Everything You Always Wanted to Know About Serverless AI But Were Too Afraid to Ask

Your N. Here
Your Institution

Second Name
Second Institution

Abstract

Accelerating generative AI adoption has spurred the expansion of datacenters, who amass troves of GPUs, DRAM, and SSDs in an effort to feed emerging resource-hungry AI workloads. The serverless cloud model offers a path to boost the resource efficiency of on-demand applications, enabling hardware sharing between tenants and applications — however, the suitability of emerging AI workloads on serverless remains insufficiently explored. We survey the state-of-the-art in serverless hosting for text based generative AI and find that: (1) despite many advancements in serverless LLM hosting, heavy model loading and initialization processes still introduce significant startup latency. (2) Agentic AI workloads have not yet been characterized under the serverless context. We propose a deployment scheme for agentic workloads that is tailored for serverless, accompanied with pre-warming policies that minimize idle resource footprint and startup latencies. This paper outlines our vision for promising research directions in serverless agents.

1 Introduction

The growth of generative AI workloads has triggered a colossal shift in computing demand. Generative AI is incredibly resource intensive — requiring large accelerators, loads of memory, and vast storage capacity. Additionally, the adoption of AI workflows in industry is accelerating, particularly through the use of text-based agentic applications [1–3]. To meet the large resource requirements of AI demand, cloud providers are scaling up by building new datacenters and stockpiling hardware [4]. This shock has manifested in global supply shortages for GPUs, DRAM, and SSDs [5, 6]. It is clear that to meet the growing demand of AI, we need serving strategies that can more efficiently manage the hardware underlying emerging AI workloads.

Most AI workloads are executed in the cloud [7–9], where deployments can fall into one of two categories: always-on or serverless. In the always-on model, software runs continuously regardless of request traffic. In contrast, serverless deployments run only when needed, invoked in response to incoming requests [10]. The serverless model allows hardware to be shared across workloads and tenants, reducing the waste of precious compute resources. Additionally, serverless users gain the added benefit of flexible billing, paying only for the resources consumed.

Still, a gap remains in assessing the suitability of AI workloads to the serverless model. The AI workloads behind generative AI are diverse: spanning LLM inferences, model training, fine tuning, agentic workloads, semantic data stores and more. To fit serverless deployment, a workload should meet two criteria: (1) requests should arrive intermittently, a constant demand would not benefit from serverless scaling. (2) The deployment should be packageable and able launch new instances on demand. We focus on LLM inferences and agentic workloads, as they dominate current AI deployments [1–3] and also show potential for request serving with ephemeral serverless backends.

Large closed weight models such as ChatGPT [11], Gemini [12], and Claude [13] see continuous demand, and would not benefit from serverless scale-to-zero mechanisms. However, we view recent trends in using small fine tuned models for narrowly scoped tasks driving the deployment of many custom, less frequently invoked LLMs [14, 15]. Small to mid-sized models can deliver similar performance for domain specific tasks at a fraction of the size, making them far more efficient than a large generalist model. Adapting these LLM deployments to the serverless model presents an opportunity for hardware sharing, however LLM inferences prove challenging to efficiently deploy under serverless. On-demand loading of model weights can cause significant cold-start delays, where requests must wait on standby for model initialization before they are serviced. Additionally, the large request footprint of each individual model can cause GPU contention during periods of high load. We survey recent work that proves the viability of serverless LLM inferencing [16–23], however we find that model cold starts remain a challenge.

In contrast to LLM inferencing, the suitability of agentic workloads to serverless remains largely unexplored — despite being a dominant component of current AI cloud deployments [2, 3]. Agents are distinct from traditional serverless workloads in a number of ways; they rely on heterogeneous compute while exhibiting non-deterministic and long running execution times. We present a wholistic characterization of agentic workloads, exploring request patterns, resource footprints, and execution overheads over representative agent architectures to inform the development of serverless policies. Recent work on agent serving leverages application level workflow structure to optimize LLM inference serving [24–27]. We propose extending this workflow structure into modular agent deployments that are more amenable for serverless infrastructure.

In this paper we contribute the following:

- We summarize the state of the art in serverless LLM hosting, covering an array of cold start mitigation techniques and serving density optimizations.
- We perform a characterization of representative agentic workloads, guiding the design of serverless policies.
- We present practical serverless deployment scenarios for agents and illustrate hidden challenges in serving agentic workloads.

2 State of the Art

2.1 Traditional Serverless

Through revisiting the decade of innovation in the traditional serverless domain, we can envision solutions to parallel problems in serverless AI hosting. Serverless computing has become an immensely well established business model offering flexible pay-as-you-go compute [10, 28–31]. Custom functions are uploaded by clients and cloud providers automatically handle resource provisioning and instance scaling as requests arrive. The sharing of underlying compute resources between function instances and tenants allows for greater serving efficiency, but also exposes challenges for function isolation and request response times. Requests must be serviced within a specified time frame determined by the provider's Service Level Objectives (SLO), but ideally these requests are handled with a minimum number of machines to most efficiently use the available compute resources. Hence, serverless systems crucially rely on instance placement and autoscaling policies to schedule function instances as requests stream in.

A circular dilemma arises in the serverless paradigm: the long-tail clients that benefit the most from flexible billing are ironically the most challenging to effectively serve. Before a request can be serviced, both the serverless runtime and client function must be loaded and initialized. However, function instances that are infrequently invoked will not stay loaded, leading to cold start delays as requests wait for instances to spin up.

The cold-start problem has inspired a large number of widely accepted policies to handle function loading and scaling. Since requests feature bursty arrival patterns, keeping instances warm in DRAM for a keep-alive timeout can obviate subsequent cold starts [32, 33]. Opportunistic pre-warming can be used to predictively load instance data before requests arrive [22, 34]. Additionally, more functions can be kept warm in DRAM through memory deduplication between instances [32, 35, 36].

Alternatively, cold start latency can be attacked head-on by optimizing function loading. Function virtualization between tenants is traditionally provided by containers, but lightweight virtualized language runtimes can be used instead to circumvent heavy container initialization costs [31, 37, 38].

2.2 Anatomy of an LLM Inference

LLM inferences are heavyweight, complicated operations that play a central role in generative AI serving. Understanding the internal structure of an inference is foundational to many systems optimizations for serving LLMs. We divide an inference into 3 stages: initialization (which is performed once at model load time), prefill, and decode — the latter two of which occur on every inference.

Initialization Before inference can start, the inference engine must load model weights into VRAM. This process is costly, as model weight files are large, reaching hundreds of gigabytes at the top end. In addition, the inference engine initializes two more optimizations. The first is the capturing of a CUDA graph, which defines the command sequence of kernels launched by the CPU during inference. Capturing the CUDA graph allows kernels to be launched on the GPU, eliminating the communication bottleneck between the CPU and GPU [39, 40]. The second optimization is the preallocation of VRAM for the KV cache.

Prefill The prefill stage ingests the prompt, builds the KV cache, and predicts the first token of the response. This stage is compute bound, as the entire prompt is processed in parallel. The KV cache is an essential datastructure for LLM inferences, storing intermediate state that would otherwise require duplicate computation during each forward pass of the decode stage.

Decode The decode stage is the autoregressive process by which each token is generated. Each step takes the last token, queries the KV cache, predicts the next token, and updates the KV cache until an termination token is generated. This stage is memory bandwidth bound, and determines the throughput of the inference.

inference figure breakdown of stages

2.3 Serverless LLMs

As the community around open weight models has matured, the demand for custom cloud inference solutions has emerged. In 2025 alone, the number of models hosted on Hugging Face doubled [15]. This trend reveals a broad interest in open weight LLMs, however hosting a large catalog of LLMs remains challenging. A serverless system that hosts inferences for custom LLMs does not have the luxury of fitting warm model instances in GPU memory. At production scale, there can be thousands of custom models to serve [17], with many experiencing sparse request patterns [41–43]. The challenge of building serverless LLM infrastructure that mitigates cold-starts and efficiency utilizes compute has seen a lot of recent attention. We provide an overview of recent work in serverless LLMs:

Model Loading. Loading model parameters into GPU memory is the single largest contributor to cold start latency for large LLMs. The cost of shipping model weights into GPU memory prior can easily dwarf the cost of a single inference. Model loading times can be partially mitigated through effective management of a GPU node's deep storage hierarchy. Prior to inference, a model checkpoint may live in VRAM, DRAM, on disk, or in remote storage; each storage tier varies by an order of magnitude in model loading speed [17].

SERVERLESSLLM [16] routes requests to nodes where model checkpoints are local, avoiding costly model loading from remote storage. When loading from remote storage is necessary, HYDRASERVE [20] provides an optimization through aggregating multiple peers' network bandwidth. Model checkpoints are distributed among peer nodes, and new instances are placed on nodes that minimize network contention.

High bandwidth GPU-GPU links are a powerful mechanism for moving model weights that are already present within one accelerator's memory. These links can be used for rapid horizontal scaling in response to request bursts [17, 44], achieving over 10× speedup for model loading at 72 billion parameters. Alternatively, high bandwidth links can be used to migrate inferences to create space for new instances to be loaded from local DRAM [16].

The current state of model cold starts: unless a model is already loaded in VRAM, or on a GPU's peer node (high-bandwidth network topology dependent), cold start times are still highly punitive.

table of cold start times across model sizes and where it is loaded from

Model Initialization. LLM inference engine initialization lies on the cold-start path for newly loaded models. MEDUSA [40] targets this initialization overhead with materialization, delegating a portion of the initialization to offline preprocessing. MEDUSA introduces optimizations to remove dynamic memory management during KV cache creation, and further parallelizes this stage alongside model loading. Similarly, CUDA graphs are consistent in structure between consecutive model loads, but still hold runtime dependencies on dynamically determined data pointer values. MEDUSA addresses non-determinism by inserting a shim layer to abstract runtime dependent kernel parameters.

Pre-warming. Serverless orchestrators can proactively load instance data into DRAM in anticipation of future requests, drastically reducing container loading overhead. This practice, called pre-warming, is an established, industry recognized technique in traditional serverless systems to mitigate cold starts. Naturally, serverless LLMs have also adopted pre-warming.

DEEPSERVE [17] implements pre-warming natively by predictively loading up to 1.5TB worth of models to DRAM. Unfortunately for LLM inferences, even if the model is present in DRAM, loading to GPU memory can still take significant time. Many prior works assume DRAM pre-warming as an implicit starting point for cold start optimizations [18, 19, 23, 40].

INSTAINFER [22] surpasses pre-warming and proposes preloading model data directly to the GPU. INSTAINFER does not evaluate against LLMs, but the technique shows promise for other ML inference workloads.

Serving Density. The heterogeneous nature of LLM inference creates a significant imbalance in resource utilization. LLM inferences compete for GPU time, leaving CPU compute underutilized. Harvesting this spare compute is as an opportunity to increase serving capacity in serverless systems. SLINFER [19] demonstrates that CPUs equipped with matrix accelerators (Intel AMX) can serve small to mid sized LLMs without violating SLOs.

CPU compute is not the only underutilized resource, the memory bandwidth heavy nature of LLM inference leaves plenty of GPU compute headroom [45]. WAGES explores GPU sharing through NVIDIA Multi-Process Service (MPS), partitioning GPUs into multiple instances for simultaneous inference of multiple models [21]. Additionally, GPUs expose fine grained frequency controls, which WAGES leverages to minimize energy consumption during LLM execution.

Another angle for optimization lies in the duplication of parameters between models. In particular, fine tuned models derived from the same base model share over 99% of parameters [18]. SERVERLESSLORA [18] leverages this to isolate the loading stages of base models from their fine tuned parameters. By keeping base models loaded in GPU memory, additional inferences can be served by just loading the fine tuned layers, reducing cold start delays and increasing deployment density. PIPEBOOST [23] takes this idea further, and allows for inference to start on the GPU in parallel with the loading of the fine tuned layers.

2.4 Agent Serving

Agentic workloads can vary greatly in architecture and capabilities. For our purposes, we define agents to be any logic that encapsulates and extends an LLM inference. This can include tool calling and actions but does not strictly require it. Some agents follow a predefined static path, while others have dynamic and recursive workflows dependent on the input query. Additionally the resources used by agents can span a large domain of complexity and resource intensity. An agent may perform simple web searches over the network, invoke approximate nearest neighbor queries into a vector database, or spin up a local container for code execution.

Existing LLM serving systems optimize for single-shot inference, overlooking the broader application context available in agentic workflows. Many works have explored how application context can be effectively utilized from the inference serving perspective, where a model is already initialized and fielding many inference requests.

Decomposing agentic workloads into their component stages reveals scheduling opportunities to more effectively use underlying hardware. The stages of agentic workflows can be effectively captured as a directed acyclic graph (DAG), where nodes perform inferences, execute tools calls, or wait on user input. Several works leverage an agent's DAG to schedule inferences, improving overall response times through batching highly-parallel inductive phases and boosting the priority of final deductive inferences [26, 46, 47].

Agentic workloads feature interdependent LLM calls, where significant portions of prompts are reused, allowing for intermediate inference results to be shared across inferences [26,27,46–48]. Context is reused across both intra-request inferences, as task specific context overlaps between subsequent workflow stages, and also across request boundaries, as agents reapply system prompts to handle new user tasks. CORTEX advocates for stage isolation, routing all inferences of each stage to dedicated inference engines, implicitly improving KV cache hit rate [27]. Alternatively, KV cache reuse can be treated as a first class citizen; HELIUM analyzes workflow stages for prompt similarity and collocates similar inferences on the same engine [26]. Additionally, HELIUM implements a proactive caching strategy that precomputes KV cache data for the static portion of agent prompts, these results are cached globally and proactively loaded into GPU memory as the inference engine fields similar requests.

3 Characterizing Agentic Resource Footprints

To design effective systems to support agentic workloads, we first need to understand their shape. A central challenge in characterizing agents is the diversity in their size and complexity. Static RAG (Retrieval Augmented Generation) workflows may only invoke a single inference with a small amount of CPU orchestration, while coding agents can exhibit long executions with stateful tool actions. We present a case study over three representative agentic workloads, chosen to cover a breadth of workflow archetypes:

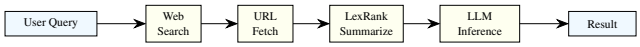

Figure 1: Langchain RAG workflow graph.

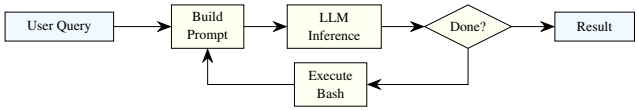

Figure 2: Mini SWE agent workflow graph.

Figure 3: GPT Researcher workflow graph. Not drawn yet :-(

Langchain RAG 1 A static question answering RAG workflow. The user query is passed to a web search API that returns a list of relevant pages, the agent then fetches the content from each page and uses LexRank [49] to extract key sentences. The summarized content is passed into a single LLM inference to generate the answer.

Mini SWE agent 2 A linear iterative workflow for solving coding tasks. The execution loop takes a query and prompts an LLM to either generate bash code or exit at each step. The bash code is then executed and the LLM is prompted again with the execution results. This is a simple example of a ReAct style agent architecture, where agents take successive reason, act, and observe steps to achieve some task [50].

GPT-Researcher 3 A multi-agent workflow for generating research reports. This is an example of both the scatter-gather architecture and multi-agent execution. A user query is processed by a planning agent, which generates subtopics for multiple researcher agents. Each researcher agent compiles data through a web search API, content fetching, and summarization loop. Finally, a writing agent compiles the results of each researcher into final report.

Our evaluation is performed on a single node equipped with an A100 GPU (80GB VRAM), two AMD 7343 16-Core Processors and 1TB DDR4 DRAM.

TODO: discuss evaluation results

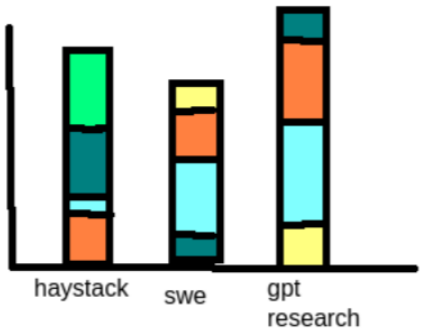

Figure 4: Execution times of agentic workloads broken down by inferences and tool use.

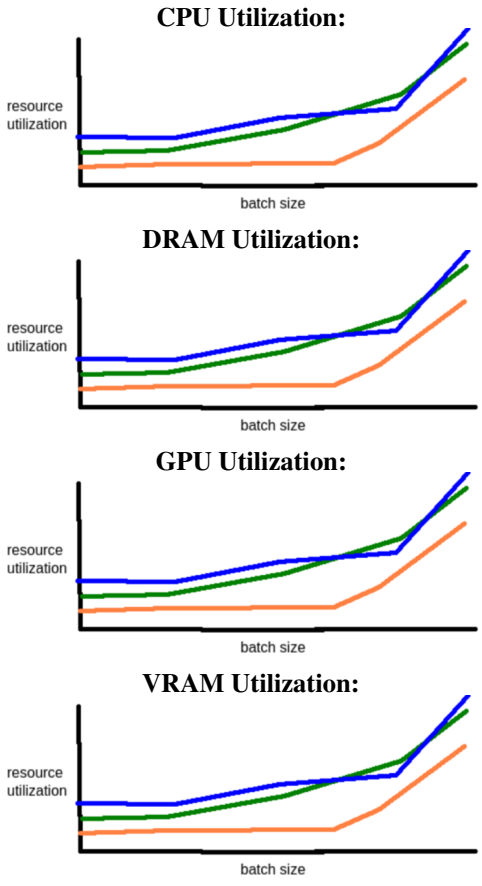

Figure 5: Resource usage of agentic workloads as batched requests increase.

4 A Vision For Serverless Agents

Agents are the dominant workload driving AI demand; improving the resource efficiency of agents under the serverless model has potential for mitigating the broader resource footprint of AI. However, agents provide a number

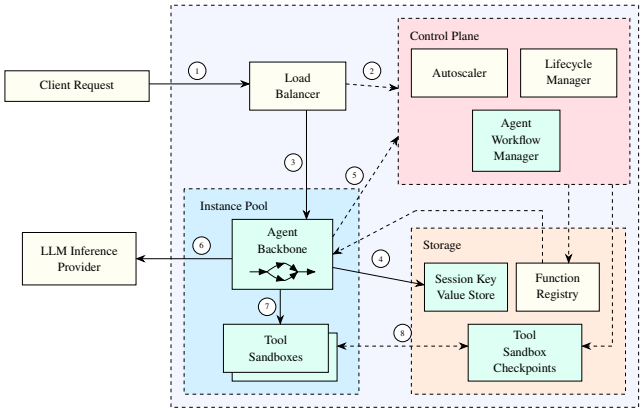

Figure 6: Design overview for our proposed modular serverless agent deployment. The light green components are additions to existing serverless infrastructure. We label interactions within the system: solid lines denote the path of client requests and dashed lines represent infrastructural interactions. (1) Client request enters the serverless system, and is routed by the load balancer. (2) The load balancer queries the serverless control plane to inform instance startup and scaling. (4) Agent backbone retrieves session history from the storage layer. (5) As the agent executes, it updates the Agent Workflow Manager with current stage progress. (6) Inferences are routed to an external LLM provider. (7) Stateful tool invocations are routed to independent tool sandboxes. (8) Tool sandboxes are checkpointed and restored according to Agent Workflow Manager policy.

of challenges that differ from traditional serverless workloads. For one, agents can be long running, with execution times on the order of minutes that amplify total resource consumption compared to high turnaround tasks. Additionally, agents are stateful; context can persist over subsequent requests and agent executions may rely on stateful actions within tool environments.

We propose the following concepts to amend agents towards practical serverless deployment: (1) **Modular Deployment**: agent workloads have a widely recognized substructure of LLM inferences and tool calls. Deconstructing agent workflow stages into isolated function instances grants flexibility to the serverless orchestrator for instance scaling. (2) **Serverless Agent Framework**: deploying agents on serverless systems requires application level changes. We introduce a programming model designed to minimize porting effort from popular agent frameworks while also enabling efficient serverless scaling between agent components. (3) **Session State Management**: to manage the high degree of session level state between agent requests, we propose a solution for checkpointing and restoring stateful tool sandboxes within the serverless context.

4.1 Monolithic vs Modular Deployments

An agent can be deployed in a serverless system today just as any other function, through isolating CPU orchestration and tooling logic. Serverless agents may make external API calls for subtasks that require heterogeneous compute or persistence: LLM inference through paid token APIs [11–13], custom models hosted on a serverless LLM platform [17, 51–53], vector databases, document stores, or any number of external resources. We refer to this methodology, where all workflow stages are bundled into one package, as the *monolithic deployment*. All serverless scheduling decisions for instance loading, keep-alive, and horizontal scaling must operate on the entire agent's container.

Alternatively, serverless agent deployments can be deconstructed into each workflow's component stages for greater resource efficiency. The representation of agentic workloads as control flow graphs (CFGs) is well supported in agent serving literature [26, 46, 47], where each node represents an LLM inference or tool execution. LLM inferences are inherently decoupled from monolithic deployments through API calls to LLM serving systems. We advocate for additionally decoupling agent orchestration logic from tool calling sandboxes, resulting in a fully *modular deployment* 6. We discuss tool sandboxes and the handling of tool side effects in 4.3. The agent orchestration backbone is left the responsibility for assembling prompts, launching inferences, parsing LLM responses, and invoking tool calls. The goal of modularizing deployment is to enable execution with the minimal set of resources active at once, aligning agent deployments with the objectives of the serverless cloud model.

Isolating agent workflow components opens possibilities for more efficiently handling serverless instance scaling from zero. Consider a keep-alive policy that selectively keeps the agent backbone warm in DRAM: when a request is fielded, the orchestration layer executes immediately while latter tool environments load in parallel. The resource footprint of a partially warm instance is dramatically reduced while approaching the performance of the monolithic deployment. A similar argument holds for container cold starts. When a request is received for a cold instance, only the orchestration layer must be initialized before function execution begins. Subsequent tool environments can load in parallel, mitigating the cold start latency compared the coarse loading of monolithic agent deployments.

The primary challenge of a modular deployment is in which stages to load and when. If workflow stages are only loaded when they are accessed, the system maintains maximal resource efficiency but pays the cost of additional internal cold starts. Conversely, if all workflow stages remain loaded over the entirety of the agent's execution, then the deployment offers no efficiency gain over the monolithic deployment. We propose the addition of an *Agent Workflow Manager* into the control plane of the serverless system that manages the lifetime of each component stage relative to agent execution progress. With the context of an agent's CFG and a history of stage execution times, the agent workflow manager can selectively unload and restore components as they are needed to strike a balance between application performance and hardware efficiency.

Small Language Models Despite the industry dominance of large generalist models, small custom models show strong performance on agentic workloads at a fraction of the size [14]. The trend towards small language model (SLM) usage in agents is motivated by faster inference speeds and smaller resource footprints at competitive application performance [14]. Fine tuned SLMs are an efficient alternative for agentic subtasks that are narrowly scoped and do not require the generality of LLMs. Support for SLMs is crucial to sustainably meet today's AI demand. We argue that serverless infrastructure for agents should be designed with the deployment of custom SLMs in mind.

SLMs are a natural fit for serverless deployment; they are lightweight and application specific. A number of practical benefits are also available for agent SLMs: they are small enough for practical CPU inference [19], many models can be pre-warmed into DRAM simultaneously, and shared base model weights can be deduplicated for substantial memory savings.

Cold starts for serverless SLM deployments can be effectively mitigated through codesign with serverless agents. Modern serverless LLM platforms still face the fundamental cold-start challenge due to the heavy data movement of model weights into GPU memory. However, with the application context from agent workflow execution, the guesswork can be entirely removed from SLM scaling. The serverless orchestrator can treat SLM inferences as another modular component of an agent's deployment, proactively loading and unloading the model based on agent execution.

4.2 Serverless Agent Framework

Migrating an agent to serverless infrastructure requires application level changes. Some level of refactoring is common for migrating any workload to serverless as developers must contend with the limitations of ephemeral function instances. However, we note that the event driven nature of agentic workflows naturally suits the serverless model. We outline a programming model that complements our modular deployment strategy, enabling stage-wise loading and execution of agent workflows.

Similar to popular agent frameworks, we model each workflow stage as a function execution. Each stage takes in the output of the previous stage, executes some work, and returns intermediate state for the succeeding stages. The stages then arranged into a control flow graph (CFG) and registered

with the serverless agent framework. Stage execution is controlled by the framework, which coordinates with the agent workflow manager in the control plane for instance loading decisions. This programming model is flexible, and comfortably handles the breadth of agent diversity, from small static workflows to complex multi-agent architectures.

Agent workflow stages can fall into one of the following: agent orchestration, LLM API calls, or function tool calls. On registration with the serverless agent framework, the function is annotated as one of these types, as well as state management context that we explore in 4.3. The serverless agent framework uses these annotations to execute the deployment with the minimal required resource footprint. For example, we find that agent orchestration logic experiences a high degree of inactivity as it waits for the results of downstream executions. This inactivity is especially prevalent for scatter-gather operations where many LLM inferences are invoked in parallel, and execution cannot continue until the last inference finishes. The serverless agent framework can instead scale down the agent backbone during these operations and pass the results into the next workflow stage when they're ready.

4.3 Session State Management

We have not yet to addressed a critical design tension for deploying agents on serverless infrastructure: agents are stateful, serverless functions are stateless. This mismatch is not a fundamental design blocker, but it does need to be addressed to prevent considerable inefficiencies in stateful function serving. From the agent's perspective, state exists at the session level — exposing a turn based interface to users for iteration on a target task. We can break down the agent's state into the following: orchestration state, used by the agent backbone for prompting, and tool execution state, which captures the results of actions within the tool execution environment.

Orchestration State **By isolating the tool execution environment from the orchestration backbone through modular deployment, we remove nearly all of the session state overhead.** The orchestration backbone still requires some session-level state for agent execution: namely the LLM prompt history which captures context from prior user requests. This data can be trivially be backed by a key value store on the session ID. The history can be retrieved when a request arrives and appended to after each LLM inference.

optmizations with cloudburst & dandelion [54, 55]

Tool Sandbox A tool environment provides agents a sandbox to enact actions and observe their results, this is particularly important for ReAct style agent architectures. The level of statefulness required in a tool sandbox varies by agent implementation. Some tool invocations are entirely stateless, like the web search in a single-turn RAG agent. However, modern coding agents depend heavily on stateful environments, requiring persistent filesytem modifications and shell sessions between requests. To support these capabilities within a serverless deployment, we require a mechanism that can snapshot a running instance, and restore that instance on demand. CRIU (Checkpoint/Restore In Userspace) [**?**] provides precisely this functionality through freezing a containers execution and checkpointing its state to disk. This technique is used in AWS Firecracker [31], to checkpoint micro VM state just after initialization to minimize cold-start delays. We can use the same concept to pause and resume an agent's tool environment according to the workflow manager's policy.

KV Cache Management The KV cache is an implicit piece of session state, hidden behind the LLM inference API, but it nonetheless is essential for agent performance. Agent prompts share a large degree of context over successive inferences, and prompt caching offers a mechanism to reduce the redundant work of computing the prefill steps for shared tokens. Some closed weight LLM APIs expose a degree of control over prompt caching [56, 57] that can enable short term storage, but the flexibility is limited. In the context of serverless SLM deployments, a codesigned approach can be taken that stashes KV caches for agent instances. HELIUM proposes precomputing the static portion of system prompts at compile time and maintaining a global view over KV caches [26]. Since KV caches can grow to gigabytes in size, the tradeoff between storing to disk and recomputing remains nuanced.

5 Conclusion

We survey the state of the art for serverless hosting of generative AI inference workloads. Recent work on serverless LLMs has introduced mechanisms that dramatically improve cold-start costs, however we posit that these costs are still impactful within the context of agent workloads. Despite work on LLM serving backends for agentic applications, we find no prior literature on serverless agents. We characterize three agent workloads across a variety of popular agent architectures and propose a practical deployment methodology for serverless agents.

References

[1] Flexera. 2026 state of the cloud report. Technical report, Flexera, 2026. Accessed: April 25, 2026.

[2] Cloudera. 96% of enterprises are expanding use of AI agents. Cloudera Press Release, 2025.

[3] Mike Vizard. Survey surfaces rapid adoption of agentic AI. Techstrong.ai, 2025.

[4] Grand View Research. U.S. data center market size, share & trends analysis report by component, by type, by server rack density, by redundancy, by PUE, by design, by tier level, by enterprise size, by end-use, and segment forecasts, 2023–2030. Technical Report GVR-4-68040-170-7, Grand View Research, 2023. Accessed: 2026-04-25.

[5] Kif Leswing. Ai memory is sold out, causing an unprecedented surge in prices. CNBC, January 2026. Accessed: 2026-04-25.

[6] Francisco Jeronimo, Tom Mainelli, Bryan Ma, Ryan Reith, and Jeff Janukowicz. Global memory shortage crisis: Market analysis and the potential impact on the smartphone and PC markets in 2026. IDC Blog, December 2025. Accessed: 2026-04-25.

[7] Ashwin Kumar Kulkarni and B. Annappa. GPU-aware resource management in heterogeneous cloud data centers. *The Journal of Supercomputing*, 77(11):12458–12485, November 2021.

[8] Gartner. Gartner says worldwide IaaS public cloud services market grew 41.4% in 2021. Gartner Press Release, 2022.

[9] Virtualization and Cloud Review. Research shows cloud workloads vary by platform, but data rules. *Virtualization and Cloud Review*, 2020.

[10] Samuel Kounev, Nikolas Herbst, Cristina L. Abad, Alexandru Iosup, Ian Foster, Prashant Shenoy, Omer Rana, and Andrew A. Chien. Serverless computing: What it is, and what it is not? *Communications of the ACM*, 66(9):80–92, September 2023.

[11] OpenAI. ChatGPT. https://chatgpt.com, 2025. Accessed: 2026-04-25.

[12] Google DeepMind. Gemini. https://gemini.google.com, 2025. Accessed: 2026-04-25.

[13] Anthropic. Claude. https://claude.ai, 2025. Accessed: 2026-04-25.

[14] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic ai, 2025.

[15] Francisco Ríos. Hugging face's two million models and counting. AI World, https://aiworld.eu/story/hugging-faces-two-million-models-and-counting, December 2025. Accessed: 2026-04-25.

[16] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models, 2024.

[17] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, Gengyuan Dan, Zhiyu Dong, Zhihao Ren, Changhong Liu, Tao Xie, Dayun Lin, Qin Zhang, Yue Yu, Hao Feng, Xusheng Chen, and Yizhou Shan. Deepserve: Serverless large language model serving at scale, 2025.

[18] Yifan Sui, Hao Wang, Hanfei Yu, Yitao Hu, Jianxun Li, and Hao Wang. Serverlesslora: Minimizing latency and cost in serverless inference for lora-based llms, 2025.

[19] Chuhao Xu, Zijun Li, Quan Chen, Han Zhao, Xueyan Tang, and Minyi Guo. Towards resource-efficient serverless llm inference with slinfer, 2025.

[20] Chiheng Lou, Sheng Qi, Chao Jin, Dapeng Nie, Haoran Yang, Yu Ding, Xuanzhe Liu, and Xin Jin. Hydraserve: Minimizing cold start latency for serverless llm serving in public clouds, 2025.

[21] Tianyu Wang, Gourav Rattihalli, Aditya Dhakal, Xulong Tang, and Dejan Milojicic. Wages: Workload-aware gpu sharing system for energy-efficient serverless llm serving. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC Workshops '25, page 508–515, New York, NY, USA, 2025. Association for Computing Machinery.

[22] Yifan Sui, Hanfei Yu, Yitao Hu, Jianxun Li, and Hao Wang. Pre-warming is not enough: Accelerating serverless inference with opportunistic pre-loading. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*, SoCC '24, page 178–195, New York, NY, USA, 2024. Association for Computing Machinery.

[23] Chongpeng Liu, Xiaojian Liao, Hancheng Liu, Limin Xiao, and Jianxin Li. Pipeboost: Resilient pipelined architecture for fast serverless llm scaling, 2025.

[24] Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang, and Tushar Krishna. Towards understanding, analyzing, and optimizing agentic ai execution: A cpu-centric perspective, 2026.

[25] Xinming Wei, Jiahao Zhang, Haoran Li, Jiayu Chen, Haoning Guan, Rui Qu, Maoliang Li, Xiang Chen, and Guojie Luo. Agent.xpu: Efficient scheduling of agentic llm workloads on heterogeneous soc, 2026.

[26] Noppanat Wadlom, Junyi Shen, and Yao Lu. Efficient llm serving for agentic workflows: A data systems perspective, 2026.

[27] Nikos Pagonas, Yeounoh Chung, Kostis Kaffes, and Arvind Krishnamurthy. Cortex: Workflow-aware resource pooling and scheduling for agentic serving, 2025.

[28] Google. Google cloud functions. https://cloud.google.com/functions/. Accessed: 2026-04-25.

[29] Microsoft. Azure functions serverless architecture. https://azure.microsoft.com/en-us/services/functions/. Accessed: 2026-04-25.

[30] Apache Software Foundation. Apache OpenWhisk is a serverless, open source cloud platform. http://openwhisk.apache.org/. Accessed: 2026-04-25.

[31] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA, February 2020. USENIX Association.

[32] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. On-demand container loading in AWS lambda. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 315–328, Boston, MA, July 2023. USENIX Association.

[33] Alexander Fuerst and Prateek Sharma. Faascache: keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '21, page 386–400, New York, NY, USA, 2021. Association for Computing Machinery.

[34] Zijun Li, Linsong Guo, Quan Chen, Jiagan Cheng, Chuhao Xu, Deze Zeng, Zhuo Song, Tao Ma, Yong Yang, Chao Li, and Minyi Guo. Help rather than recycle: Alleviating cold startup in serverless computing through Inter-Function container sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 69–84, Carlsbad, CA, July 2022. USENIX Association.

[35] Divyanshu Saxena, Tao Ji, Arjun Singhvi, Junaid Khalid, and Aditya Akella. Memory deduplication for serverless computing with medes. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 714–729, New York, NY, USA, 2022. Association for Computing Machinery.

[36] Wei Qiu, Marcin Copik, Yun Wang, Alexandru Calotoiu, and Torsten Hoefler. User-guided page merging for memory deduplication in serverless systems, 2023.

[37] Nicholas C. Wanninger, Joshua J. Bowden, Kirtankumar Shetty, Ayush Garg, and Kyle C. Hale. Isolating functions at the hardware limit with virtines. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 644–662, New York, NY, USA, 2022. Association for Computing Machinery.

[38] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 467–481, New York, NY, USA, 2020. Association for Computing Machinery.

[39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.

[40] Shaoxun Zeng, Minhui Xie, Shiwei Gao, Youmin Chen, and Youyou Lu. Medusa: Accelerating serverless llm inference with materialization. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, page 653–668, New York, NY, USA, 2025. Association for Computing Machinery.

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. Lmsys-chat-1m: A large-scale real-world llm conversation dataset, 2024.

[42] Yuxing Xiang, Xue Li, Kun Qian, Wenyuan Yu, Ennan Zhai, and Xin Jin. Servegen: Workload characterization and generation of large language model serving in production, 2025.

[43] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize llm serving systems, 2025.

[44] Minchen Yu, Rui Yang, Chaobo Jia, Zhaoyuan Su, Sheng Yao, Tingfeng Lan, Yuchen Yang, Zirui Wang, Yue Cheng, Wei Wang, Ao Wang, and Ruichuan Chen. λscale: Enabling fast scaling for serverless large language model inference, 2026.

[45] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex

Vaughan, Amy Yang, et al. The llama 3 herd of models, 2024.

[46] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable, 2024.

[47] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for llm agents as general programs, 2025.

[48] You Peng, Youhe Jiang, Wenqi Jiang, Chen Wang, and Binhang Yuan. Hexgen-flow: Optimizing llm inference request scheduling for agentic text-to-sql, 2026.

[49] Günes Erkan and Dragomir R. Radev. Lexrank: Graph-based lexical centrality as salience in text summarization. *CoRR*, abs/1109.2128, 2011.

[50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.

[51] Amazon Web Services. Amazon bedrock. https://aws.amazon.com/bedrock/, September 2023. Generally available September 28, 2023. Accessed: 2026-04-25.

[52] Oracle. OCI generative AI. https://www.oracle.com/artificial-intelligence/generative-ai/generative-ai-service/, January 2024. Generally available January 23, 2024. Accessed: 2026-04-25.

[53] Anyscale. Anyscale: A unified AI compute platform. https://www.anyscale.com, 2019. Founded 2019. Accessed: 2026-04-25.

[54] Tom Kuchler, Pinghe Li, Yazhuo Zhang, Lazar Cvetković, Boris Goranov, Tobias Stocker, Leon Thomm, Simone Kalbermatter, Tim Notter, Andrea Lattuada, and Ana Klimovic. Unlocking true elasticity for the cloud-native era with dandelion, 2025.

[55] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. Cloudburst: stateful functions-as-a-service. *Proc. VLDB Endow.*, 13(12):2438–2452, July 2020.

[56] OpenAI. Prompt caching. https://developers.openai.com/api/docs/guides/prompt-caching. OpenAI API Documentation. Accessed: 2026-05-15.

[57] Anthropic. Prompt caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching. Claude API Docs. Accessed: 2026-05-15.